

My SQL Project

Name of the Project: Analysis of Movies Database



Insights of the Project:

Here are some valuable insights that could be derived from a movies database for an SQL project. These insights cover popular trends, user behaviour, ratings, and genre analysis, among others. Each of these insights can be achieved by writing SQL queries that analyse data in various ways.

1. Top-Rated Movies by Genre

- Identify the highest-rated movies within each genre. This can be useful to see which movies stand out in specific genres.
- **Example Query:** Use GROUP BY Genre and ORDER BY Rating Value to get top-rated movies in each genre.

2. Most Popular Actors

- Find actors who have appeared in the most movies, indicating their popularity and frequency of casting.
- **Example Query:** Count movies per actor and order by count to find top actors.

3. Average Rating by Genre

- Calculate the average user rating for each genre, which can reveal which genres are most appreciated by users.
- **Example Query:** Use AVG (Rating Value) and GROUP BY Genre.

4. Most Watched Movies

- Find the movies that have been watched the most times by users, which helps in understanding viewer preferences.
- **Example Query:** Count Watched entries per movie and order by count.

5. Most Active Users

- Identify users who have watched the most movies, highlighting the most engaged users on the platform.
- **Example Query:** Count Watched entries per user and order by count.

6. Directors with the Highest Average Ratings

- Determine which directors have the highest average ratings across all their movies. This can reveal which directors consistently produce well-received content.
- **Example Query:** Join Movie and Director, then calculate AVG (Rating Value) for each director.

7. Trends in Movie Releases Over Time

- Analyse the number of movies released per year or decade, showing trends in movie production over time.
- **Example Query:** Group by Release Year to count the movies per year.

8. User Rating Patterns

- Analyse the rating habits of users, for example, seeing if certain users tend to give higher or lower ratings on average.

- **Example Query:** Calculate AVG (Rating Value) per user and check for patterns.

9. Most Popular Genres by Region

- If the database includes user locations, identify which genres are most popular in different regions or countries.
- **Example Query:** Join User and Genre with a count of Watched or Rated grouped by region.

10. Movies with Consistently High Ratings Over Time

- Identify movies that have maintained high ratings since their release, indicating classic or enduring appeal.
- **Example Query:** Track changes in ratings over time for each movie, perhaps using a HAVING clause for consistently high ratings.

11. Average Movie Duration by Genre

- Calculate the average duration of movies in each genre, which could reveal differences in content length across genres.
- **Example Query:** AVG(Duration) grouped by Genre.

12. User Preferences for Directors or Actors

- Find which directors or actors are most liked by users based on their viewing and rating habits.
- **Example Query:** Join User, Actor, or Director with Watched or Rated, then group and order by count or average rating.

13. Peak Viewing Times or Days

- Analyse when users watch the most movies (day of the week or time of day), which can be useful for optimizing content recommendations.
- **Example Query:** Use a timestamp field in the Watched table and group by time or day.

14. Movies With Mixed Reviews

- Find movies that have high variance in ratings, indicating polarizing content.
- **Example Query:** Use STDDEV (Rating Value) to measure rating variance.

15. Correlations Between Movie Attributes and Ratings

- Look for correlations, such as whether movies with longer durations or certain genres tend to have higher ratings.
- **Example Query:** Perform analysis with multiple attributes, such as comparing Duration and Rating Value.

These insights can help provide data-driven recommendations, identify trends, and inform content strategies for the movie's platform. Each of these queries can be expanded or refined based on the database structure and specific project needs.

ER Diagram:

Entities and Attributes

1. **Movie**

- **Attributes:** Movie ID (PK), Title, Release Year, Rating, Duration, Language
- **Relationships:**
 - Has a relationship with **Genre** (many-to-many).
 - Stars in relationship with **Actor** (many-to-many).
 - Directed by relationship with **Director** (many-to-one).
 - Watched by relationship with **User** (many-to-many).

2. **Actor**

- **Attributes:** Actor ID (PK), Name, Birth Date, Nationality
- **Relationships:**
 - Stars in relationship with **Movie** (many-to-many).

3. **Director**

- **Attributes:** Director ID (PK), Name, Birth Date, Nationality
- **Relationships:**
 - Directs relationship with **Movie** (one-to-many).

4. **Genre**

- **Attributes:** Genre ID (PK), Genre Name
- **Relationships:**
 - Categorizes relationship with **Movie** (many-to-many).

5. **User**

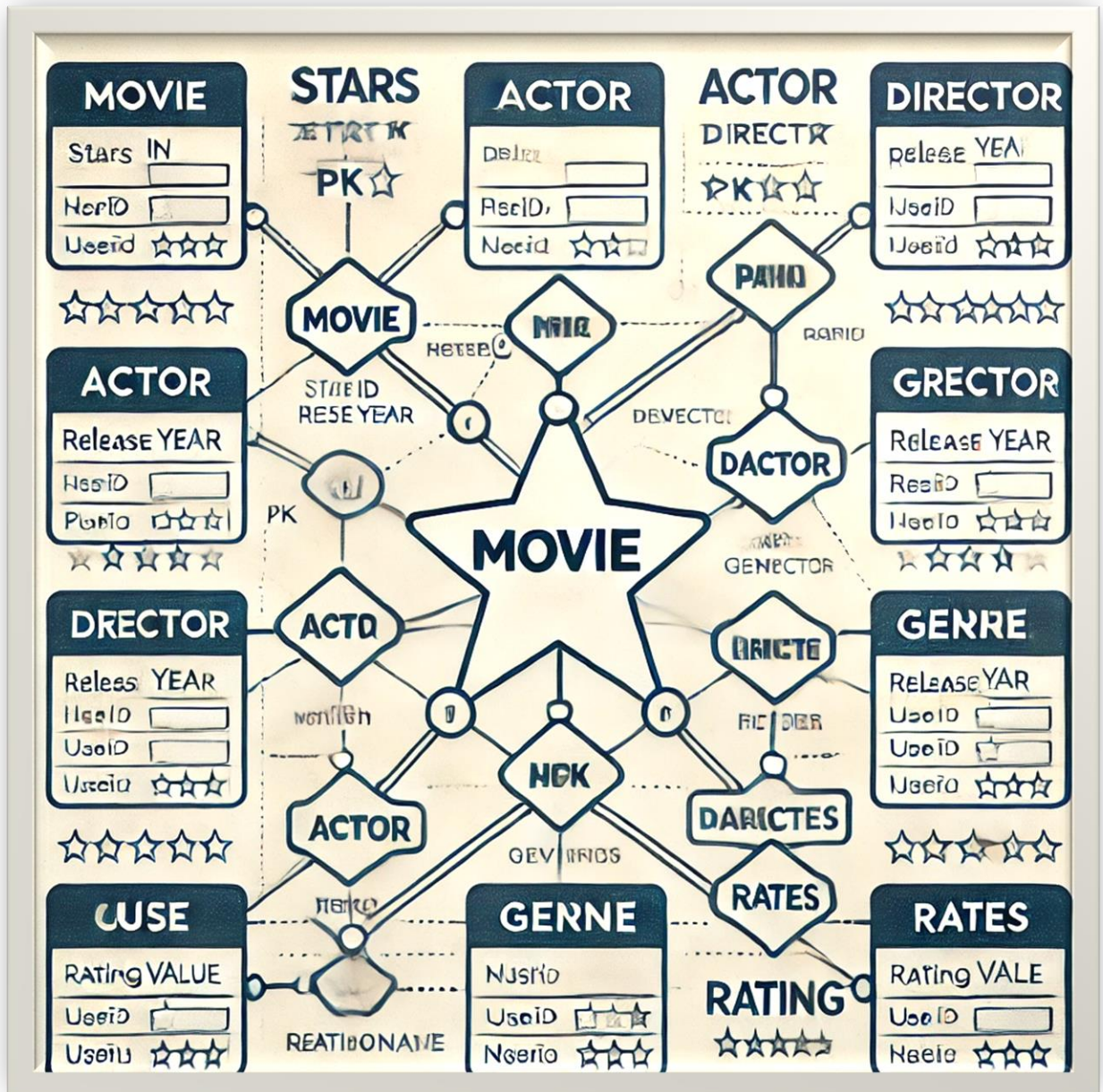
- **Attributes:** User ID (PK), Username, Email, Password
- **Relationships:**
 - Watched relationship with **Movie** (many-to-many).
 - Rates relationship with **Movie** (many-to-many).

6. Rating

- **Attributes:** Rating ID (PK), User ID (FK), Movie ID (FK), Rating Value, Rating Date
- **Relationships:**
 - Links **User** and **Movie** in a many-to-many relationship, where users rate movies.

Relationships

- **Stars In** (Many-to-Many): An actor can star in multiple movies, and a movie can have multiple actors.
- **Directs** (One-to-Many): A director can direct multiple movies, but a movie has one director.
- **Categorizes** (Many-to-Many): A movie can have multiple genres, and a genre can categorize multiple movies.
- **Watches** (Many-to-Many): A user can watch multiple movies, and a movie can be watched by multiple users.
- **Rates** (Many-to-Many): A user can rate multiple movies, and a movie can be rated by multiple users.



Based on this database we can run some Queries to analyse the data and get required output:

-- 1. Get a list of all movies with their genres.

```
SELECT m.title, g.genre_name
FROM Movies m
JOIN Genres g ON m.genre_id = g.genre_id;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
title	genre_name		
Avengers: Endgame	Action		
The Wolf of Wall Street	Drama		
Interstellar	Sci-Fi		
Inception	Sci-Fi		
The Notebook	Romance		
The Shining	Horror		
Jurassic Park	Adventure		
Se7en	Thriller		
Shutter Island	Mystery		
Harry Potter and the Sorcerer's Stone	Fantasy		

-- 2. Find all movies released after the year 2010.

```
SELECT title, release_year
FROM Movies
WHERE release_year > 2010;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
title	release_year		
Avengers: Endgame	2019		
The Wolf of Wall Street	2013		
Interstellar	2014		

```
-- 3. Retrieve all actors who starred in the movie "Inception."
SELECT a.first_name, a.last_name
FROM Actors a
JOIN Movie_Actor ma ON a.actor_id = ma.actor_id
JOIN Movies m ON ma.movie_id = m.movie_id
WHERE m.title = 'Inception';
```

Result Grid			Filter Rows:	Export: 	Wrap Cell Content: 
first_name	last_name				

```
-- 4. Get the average rating of each movie.
SELECT m.title, AVG(r.rating) AS average_rating
FROM Movies m
JOIN Reviews r ON m.movie_id = r.movie_id
GROUP BY m.title;
```

Result Grid			Filter Rows:	Export: 	Wrap Cell Content: 
	title	average_rating			
▶	Avengers: Endgame	9.00000			
	The Wolf of Wall Street	8.50000			
	Interstellar	9.10000			
	Inception	9.30000			
	The Notebook	7.90000			
	The Shining	8.70000			
	Jurassic Park	8.30000			
	Se7en	8.80000			
	Shutter Island	8.40000			
	Harry Potter and the Sorcerer's Stone	7.70000			

```
-- 5. Find the top 5 highest-rated movies.
```

```
SELECT title, rating
FROM Movies
ORDER BY rating DESC
LIMIT 5;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:	Fetch rows:
	title	rating			
▶	Inception	8.8			
	Interstellar	8.6			
	Se7en	8.6			
	Avengers: Endgame	8.4			
	The Shining	8.4			

```
-- 6. Retrieve the count of movies for each genre.
```

```
SELECT g.genre_name, COUNT(m.movie_id) AS movie_count
FROM Genres g
LEFT JOIN Movies m ON g.genre_id = m.genre_id
GROUP BY g.genre_name;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	genre_name	movie_count		
▶	Action	1		
	Adventure	1		
	Comedy	0		
	Drama	1		
	Fantasy	1		
	Horror	1		
	Mystery	1		
	Romance	1		
	Sci-Fi	2		
	Thriller	1		

```
-- 7. Find all reviews for the movie "The Notebook."
```

```
SELECT r.review_text, r.rating, r.review_date
FROM Reviews r
JOIN Movies m ON r.movie_id = m.movie_id
WHERE m.title = 'The Notebook';
```

Result Grid  Filter Rows: Export:  Wrap Cell Content: 			
	review_text	rating	review_date
▶	A touching and emotional love story.	7.9	2004-06-25

```
-- 8. List actors who appeared in more than one movie.
SELECT a.first_name, a.last_name, COUNT(ma.movie_id) AS movie_count
FROM Actors a
JOIN Movie_Actor ma ON a.actor_id = ma.actor_id
GROUP BY a.actor_id
HAVING movie_count > 1;
```

Result Grid  Filter Rows: Export:  Wrap Cell Content: 			
	first_name	last_name	movie_count

```
-- 9. Get the average movie rating by genre.
SELECT g.genre_name, AVG(m.rating) AS avg_rating
FROM Genres g
JOIN Movies m ON g.genre_id = m.genre_id
GROUP BY g.genre_name;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	genre_name	avg_rating			
▶	Action	8.40000			
	Drama	8.20000			
	Sci-Fi	8.70000			
	Romance	7.80000			
	Horror	8.40000			
	Adventure	8.10000			
	Thriller	8.60000			
	Mystery	8.20000			
	Fantasy	7.60000			

```
-- 10. List all movies along with the number of reviews they have received.
SELECT m.title, COUNT(r.review_id) AS review_count
FROM Movies m
LEFT JOIN Reviews r ON m.movie_id = r.movie_id
GROUP BY m.movie_id;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	title	review_count			
▶	Avengers: Endgame	1			
	The Wolf of Wall Street	1			
	Interstellar	1			
	Inception	1			
	The Notebook	1			
	The Shining	1			
	Jurassic Park	1			
	Se7en	1			
	Shutter Island	1			
	Harry Potter and the Sorcerer's Stone	1			

```
-- 11. Retrieve movies with a rating above 8.0 and list their genres.
SELECT m.title, m.rating, g.genre_name
FROM Movies m
JOIN Genres g ON m.genre_id = g.genre_id
WHERE m.rating > 8.0;
```

Result Grid			
		Filter Rows:	
		Export:	
		Wrap Cell Content:	
	title	rating	genre_name
▶	Avengers: Endgame	8.4	Action
	The Wolf of Wall Street	8.2	Drama
	Interstellar	8.6	Sci-Fi
	Inception	8.8	Sci-Fi
	The Shining	8.4	Horror
	Jurassic Park	8.1	Adventure
	Se7en	8.6	Thriller
	Shutter Island	8.2	Mystery

-- 12. Find the genre with the highest average rating.

```
SELECT g.genre_name, AVG(m.rating) AS avg_rating
FROM Genres g
JOIN Movies m ON g.genre_id = m.genre_id
GROUP BY g.genre_name
ORDER BY avg_rating DESC
LIMIT 1;
```

Result Grid			
		Filter Rows:	
		Export:	
		Wrap Cell Content:	
		Fetch rows:	
	genre_name	avg_rating	
▶	Sci-Fi	8.70000	

-- 13. Retrieve the top 3 longest movies, along with their genres and durations.

```
SELECT m.title, g.genre_name, m.duration_minutes
FROM Movies m
JOIN Genres g ON m.genre_id = g.genre_id
ORDER BY m.duration_minutes DESC
LIMIT 3;
```

Result Grid			
		Filter Rows:	
		Export:	
		Wrap Cell Content:	
		Fetch rows:	
	title	genre_name	duration_minutes
▶	Avengers: Endgame	Action	181
	The Wolf of Wall Street	Drama	180
	Interstellar	Sci-Fi	169

```
-- 14. Get the most reviewed movie and the number of reviews it has received.
SELECT m.title, COUNT(r.review_id) AS review_count
FROM Movies m
JOIN Reviews r ON m.movie_id = r.movie_id
GROUP BY m.movie_id
ORDER BY review_count DESC
LIMIT 1;
```

Result Grid		
		Filter Rows:
		Export:
		Wrap Cell Content:
		Fetch rows:
	title	review_count
▶	Avengers: Endgame	1

```
-- 15. Find all genres that have at least two movies rated above 8.0.
SELECT g.genre_name
FROM Genres g
JOIN Movies m ON g.genre_id = m.genre_id
WHERE m.rating > 8.0
GROUP BY g.genre_name
HAVING COUNT(m.movie_id) >= 2;
```

Result Grid	
	genre_name
▶	Sci-Fi

Conclusion:

In conclusion, developing a movie database using SQL provides a robust platform to manage and analyse vast amounts of information related to movies, actors, directors, genres, and user interactions. This project demonstrates the effectiveness of relational databases in organizing complex data structures, creating relationships among entities, and enabling efficient querying capabilities.



This Photo by Unknown Author is licensed under [CC BY](#)

